16ms 안에 그려라

올리브영 Android 개발자의 화면 성능 고민
덜 다시 그리고, 덜 계산하고, 덜 만들기

유준성 · 웰니스서비스개발팀 Android

AGENDA

오늘 이야기

01 배경 — 웹뷰·네이티브, 올리브베러

02 메인 스레드와 16ms

03 덜 다시 그리기 — Recomposition

04 덜 다시 계산하기 — remember

05 덜 만들기 — 메모리·할당

06 결과 · 다음 단계 · 정리

01 배경

웹뷰 화면과 네이티브 화면

웹뷰 (WebView)



① 프론트엔드를 웹 서버에 배포

② 앱의 웹뷰가 그 URL을 불러온다

→ 앱 업데이트 없이 바로 반영

네이티브 (Jetpack Compose)



① Kotlin 코드를 작성해 앱을 빌드

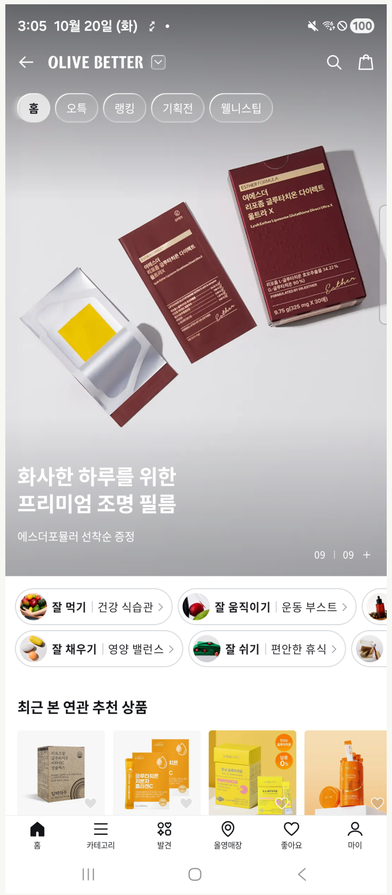
② 앱 스토어에 배포 (심사 포함)

→ 사용자가 앱을 업데이트해야 반영

01 배경

올리브베러(OB) 네이티브 지면

Jetpack Compose로 만든 네이티브 UI



화면 구성

- 메인 배너 — 자동으로 넘어가는 이미지·영상
- 상품 카드 — 좋아요 하트, 카운트다운 타이머
- 영상 카드 — 스크롤하며 자동 재생
- 로딩 스켈레톤 — 반짝이는 애니메이션
- 웹 탭 — 웹뷰 + 플로팅 버튼

증상

- 스크롤·스와이프 중 버벅임
- 애니메이션 도는 동안 프레임 드랍
- 탭 넘길 때 순간 끊김

02 개념 ① 메인 스레드

화면을 그리는 스레드는 단 하

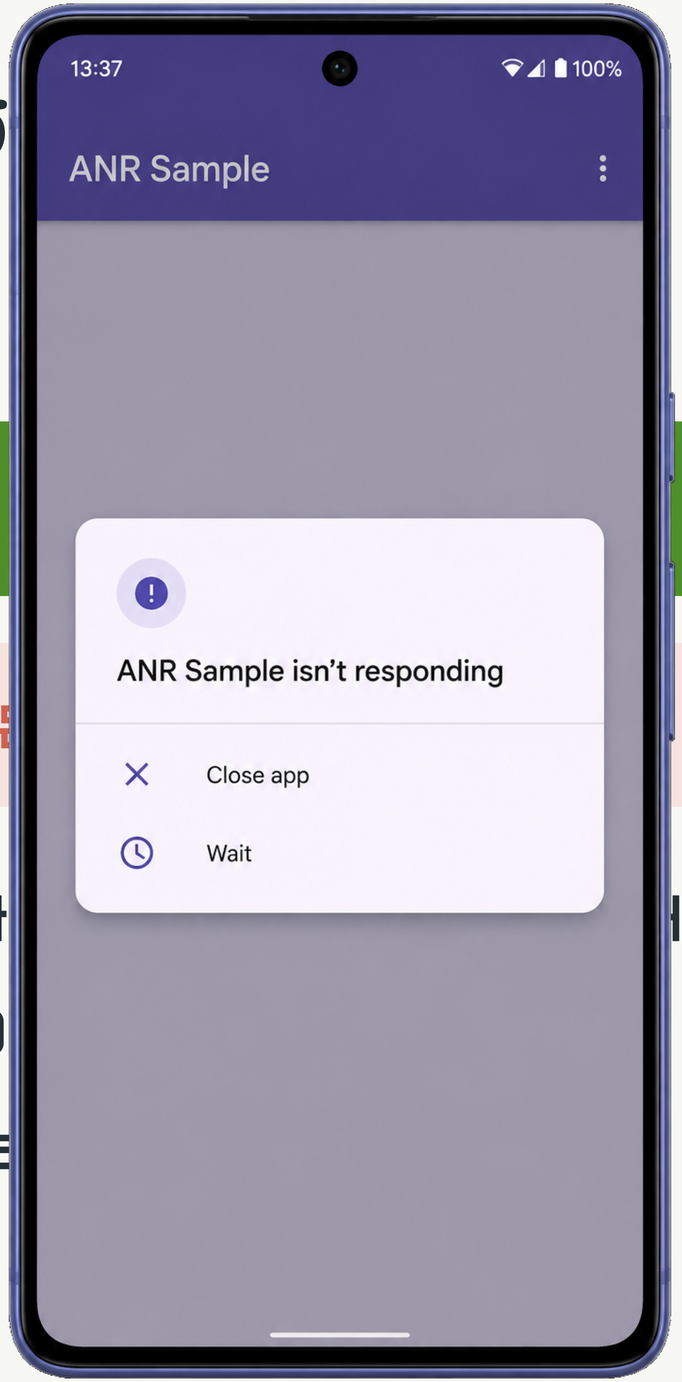
메인(UI) 스레드가 하는 일

터치 입력

그리기 명령

여기에 무거운 작업이 끼면 → 뒤의 모

- UI는 메인 스레드에서만 바꿀 수 있다
- 5초 넘게 응답이 없으면 ANR — '앱이
- API 통신·JSON 변환처럼 오래 걸리는



02 개념 ② 프레임 예산

한 프레임에 주어진 시간, 16ms

60Hz 화면	90Hz 화면	120Hz 화면
16.7ms	11.1ms	8.3ms

60Hz 화면에서 프레임이 그려지는 모습



마감까지 못 그려서 같은 화면을 한 번 더 → 버벅임 (jank)

UI 최적화 = 정해진 시간 안에 반드시 프레임을 그리는 것

02 사례 · 메인 스레드에서 내려오기

suspend 함수라고 메인 스레드를 떠나는 건 아니다

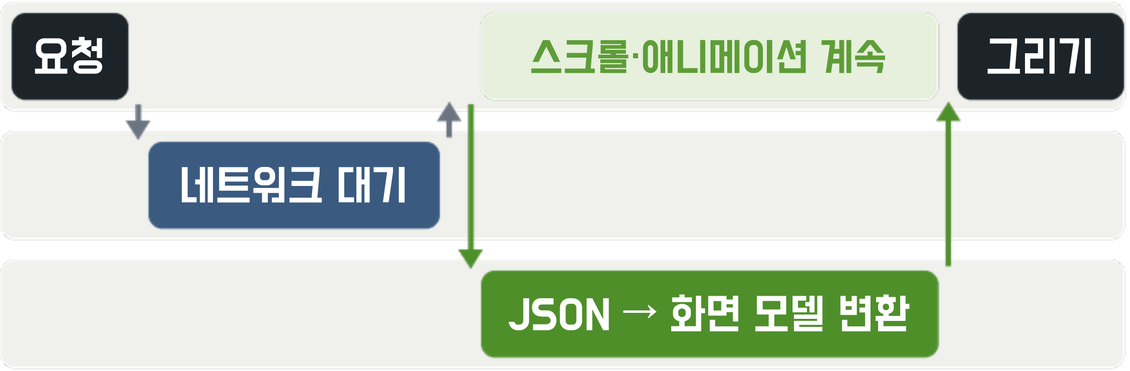
Before

변환이 메인 스레드에서 → 버벅임



After

변환은 Default에서, 메인은 계속 그린다



개선 코드

```
// JSON → 화면 모델
// 변환만 옮긴다

withContext(Dispatchers.Default) {
  mapper.toPage(response)
}
```

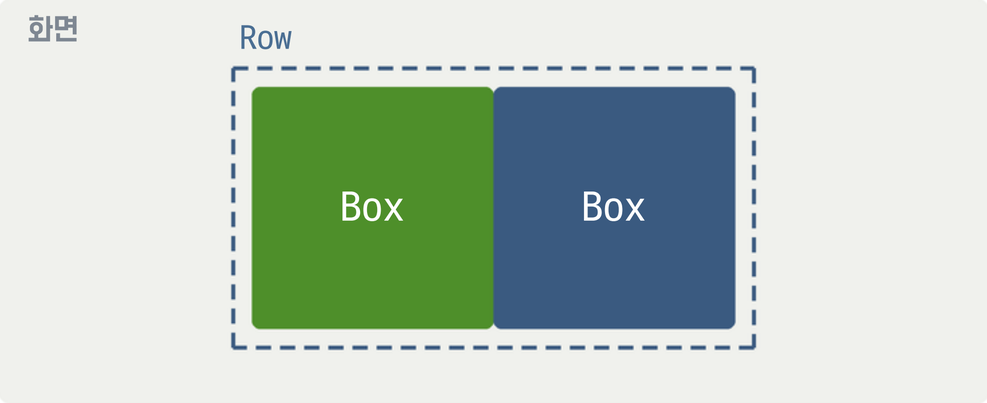
03 개념 · 선언형 UI

함수가 곧 화면이 된다

개념 예시

```
@Composable
fun TwoBoxes() {
  Row {
    Box(Modifier.size(80.dp).background(Green))
    Box(Modifier.size(80.dp).background(Blue))
  }
}
// Row = 가로로 나란히, Box = 사각형 하나
```

함수를 실행하면, 이런 화면이 나온다



Row { } → 안에 든 것을 가로로 나란히 놓는다

Box() → 사각형 하나

상태가 바뀌면, 함수가 다시 실행된다

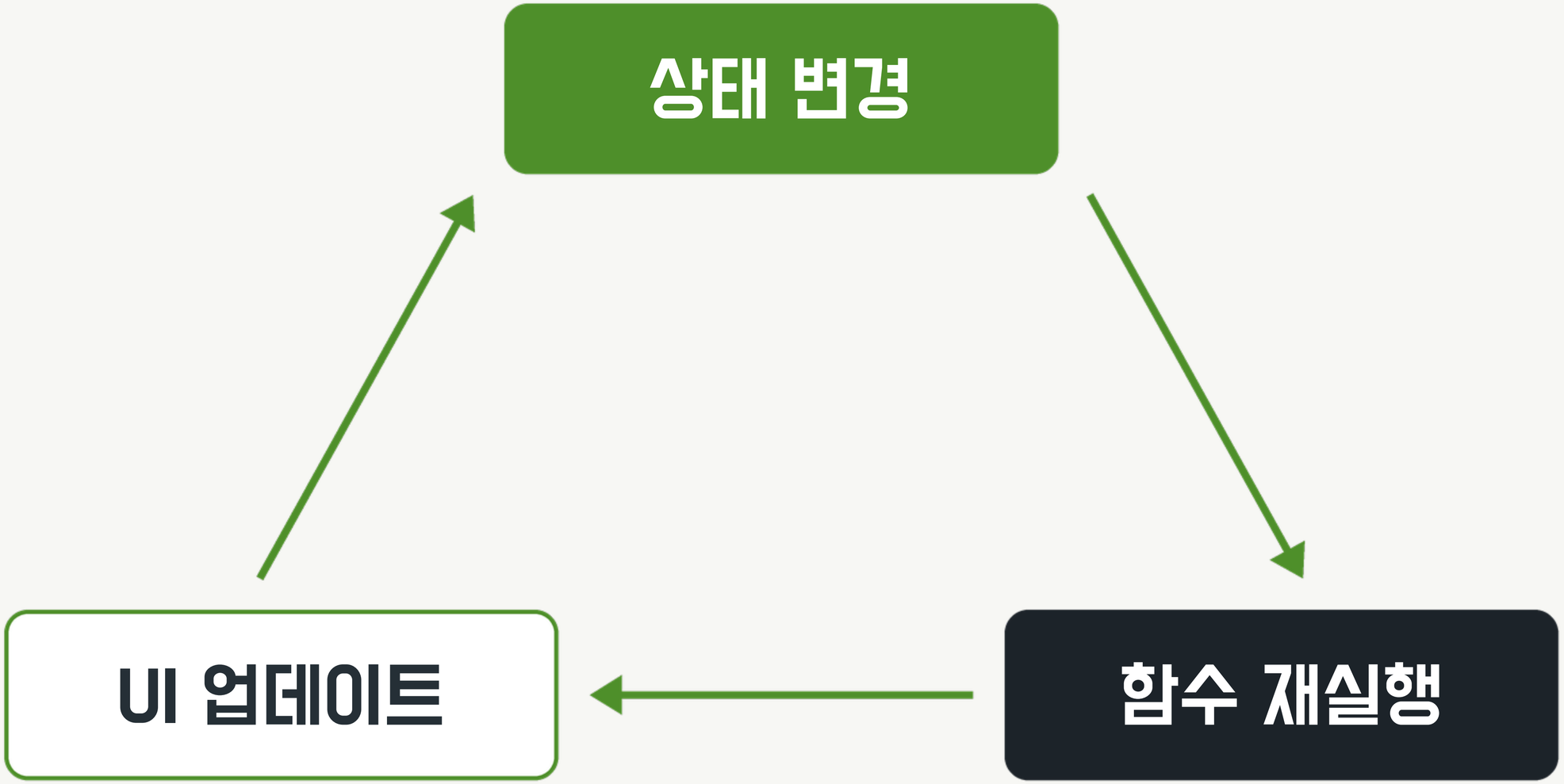
개념 예시

```
@Composable
fun Greeting() {
  var name by remember {
    mutableStateOf("올리브")
  }
  Text("안녕, $name")
}

// name = 상태
```



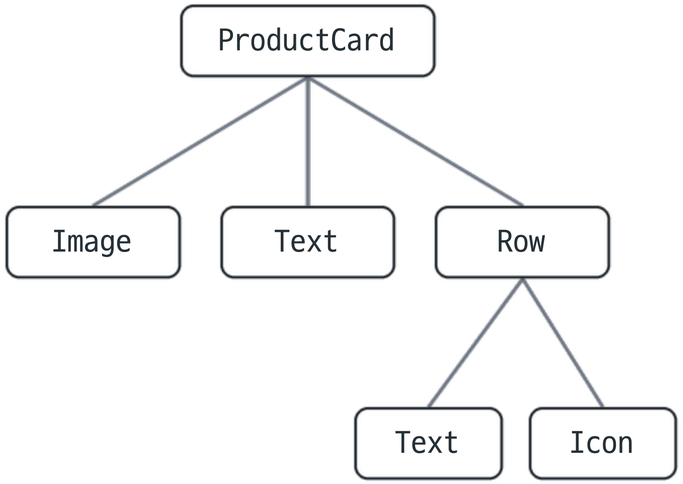
UI = f(state)



Compose는 세 단계로 한 프레임을 만든다

① Composition

무엇을 보여줄지 → UI 트리

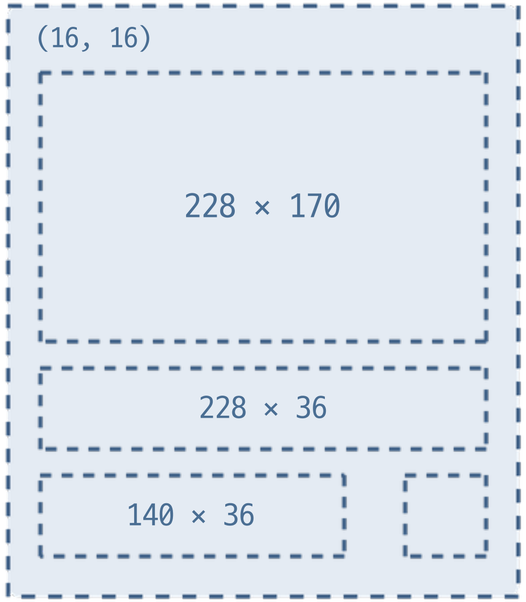


화면에 '무엇이' 있는지만 정한다

아직 크기도, 위치도, 픽셀도 없다

② Layout

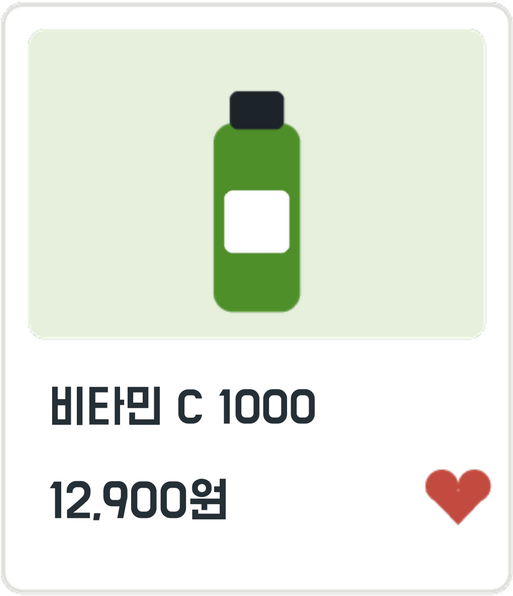
크기 측정(Measure) → 위치 배치(Place)



각 요소의 크기와 좌표가 정해진다

③ Draw

정해진 자리에 픽셀을 칠한다



우리가 실제로 보는 화면

어느 단계에서 읽느냐가 비용을 정한다

Composition
무엇을 보여줄지
함수 실행

→

Layout
크기 측정
위치 배치

→

Draw
픽셀 그리기

값을 읽은 단계부터 다시 실행된다

Composition에서 읽으면 — 세 단계 전부

Layout에서 읽으면 — Layout + Draw

Draw에서 읽으면 — 다시 칠하기만

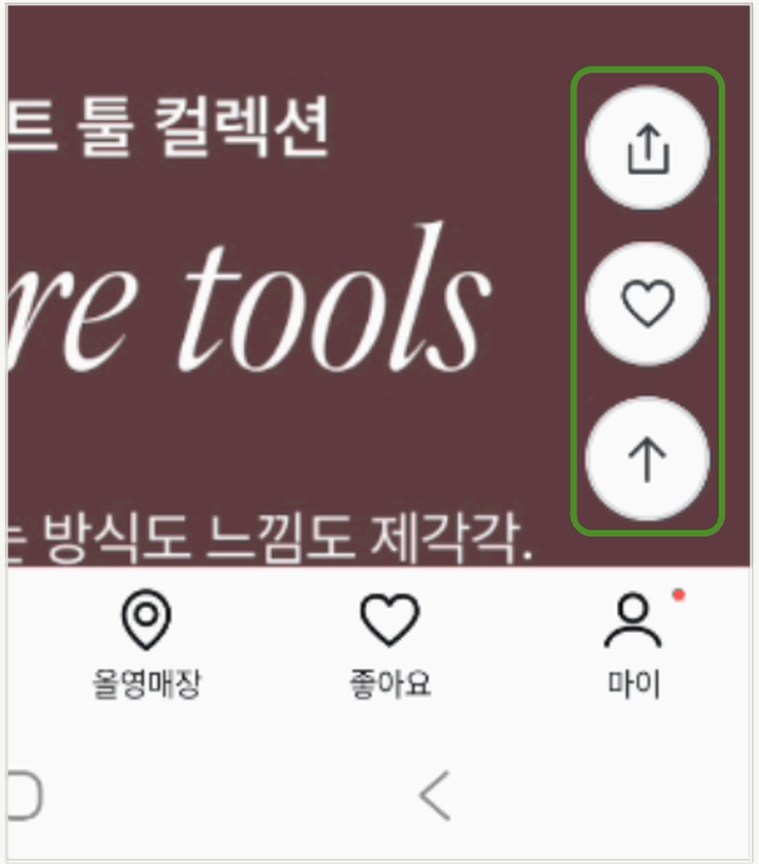
UI는 초당 60번 이상 바뀐다 → 어떤 단계에서 읽느냐가 곧 프레임당 비용

최대한 상태값을 늦게 읽되, 필요한 단계를 건너뛰면 안된다.

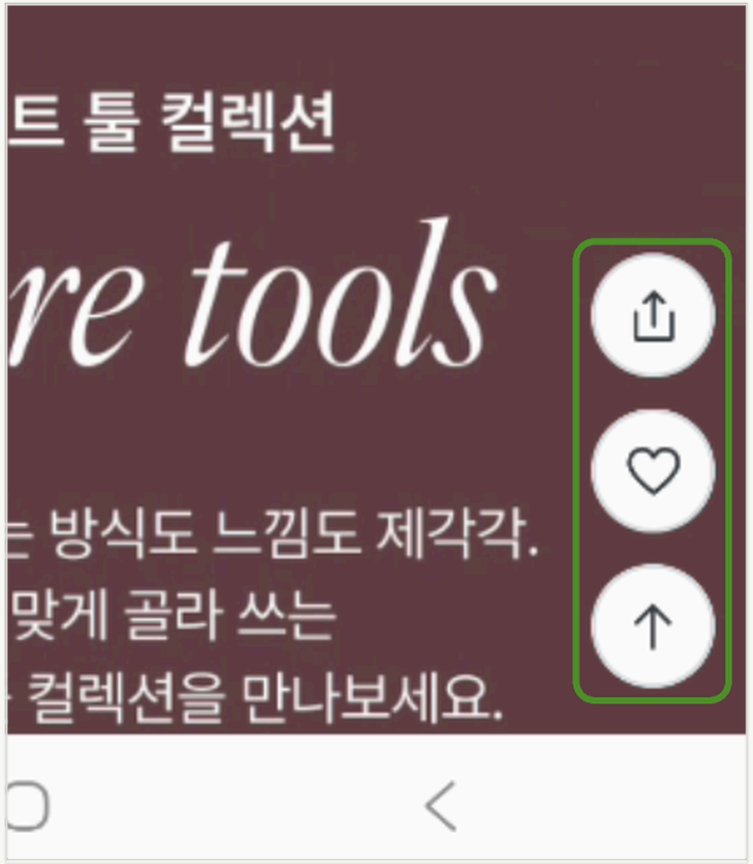
03 사례 ① Layout 단계(배치)에서 읽기

스크롤하면 플로팅 버튼이 움직인다

① Composition | ② Layout | ③ Draw



아래로 스와이프 · 버튼 올라감



위로 스와이프 · 버튼 내려감

y 좌표(상태)가 바뀐다

↓

UI 함수가 다시 실행된다

↓

버튼 위치가 바뀐다

03 사례 ① Layout 단계(배치)에서 읽기

위치만 바뀔 뿐, 버튼은 그대로 있다

① Composition | ② Layout | ③ Draw

UI 트리



y 좌표만 바뀜

위치가 바뀌어도, 트리 속 버튼은
계속 그 자리에 있다

다시 시작할 단계

① Composition
무엇이 있는지 확인
건너뛴다

② Layout
크기와 위치 결정
여기서부터

③ Draw
화면에 그리기

03 사례 ① Layout 단계(배치)에서 읽기

y 좌표는 두 번째 단계에서 읽는다

① Composition ② Layout ③ Draw

Before

```
val y = tabBarY.collectAsState().value
// ① 단계에서 읽음

Modifier.offset { IntOffset(x, y) }
```

매 프레임 다시 실행되는 단계

Composition Layout Draw

세 단계 전부

After

```
val y = tabBarY.collectAsState()

Modifier.offset { IntOffset(x, y.value) }
// ② 단계에서 읽음
```

매 프레임 다시 실행되는 단계

Composition Layout Draw

첫 단계를 건너뛴다

03 사례 ② Draw 단계에서 읽기

배너가 넘어갈 때, 옆 배너가 어두워진다

① Composition ② Layout ③ Draw





어두운 정도(상태)가 바뀐다

↓

배너 화면 전체를 다시 계산

↓

배너가 넘어갈 때마다 반복

03 사례 ② Draw 단계에서 읽기

애니메이션 값은 '그릴 때'만 읽기

① Composition ② Layout ③ Draw

매 프레임 다시 실행되는 단계

Before — Composition · Layout · Draw

세 단계 전부

After — Composition · Layout · **Draw**

다시 칠하기만

어두워지기만 하면, 트리도 위치도 그대로다

다시 실행된 횟수	변경 전	변경 후
배너 페이지 전체	50	15
배너 이미지	43	6
배너 위 텍스트	41	6
딤(어둡게) 레이어	40	3

배너 1회 전환(4.8초 구간) 동안 측정
에뮬레이터 · 2회 반복 결과 동일

03 사례 ③ 한 줄 수정

UI 트리 전체에 값을 내려 주는 장치

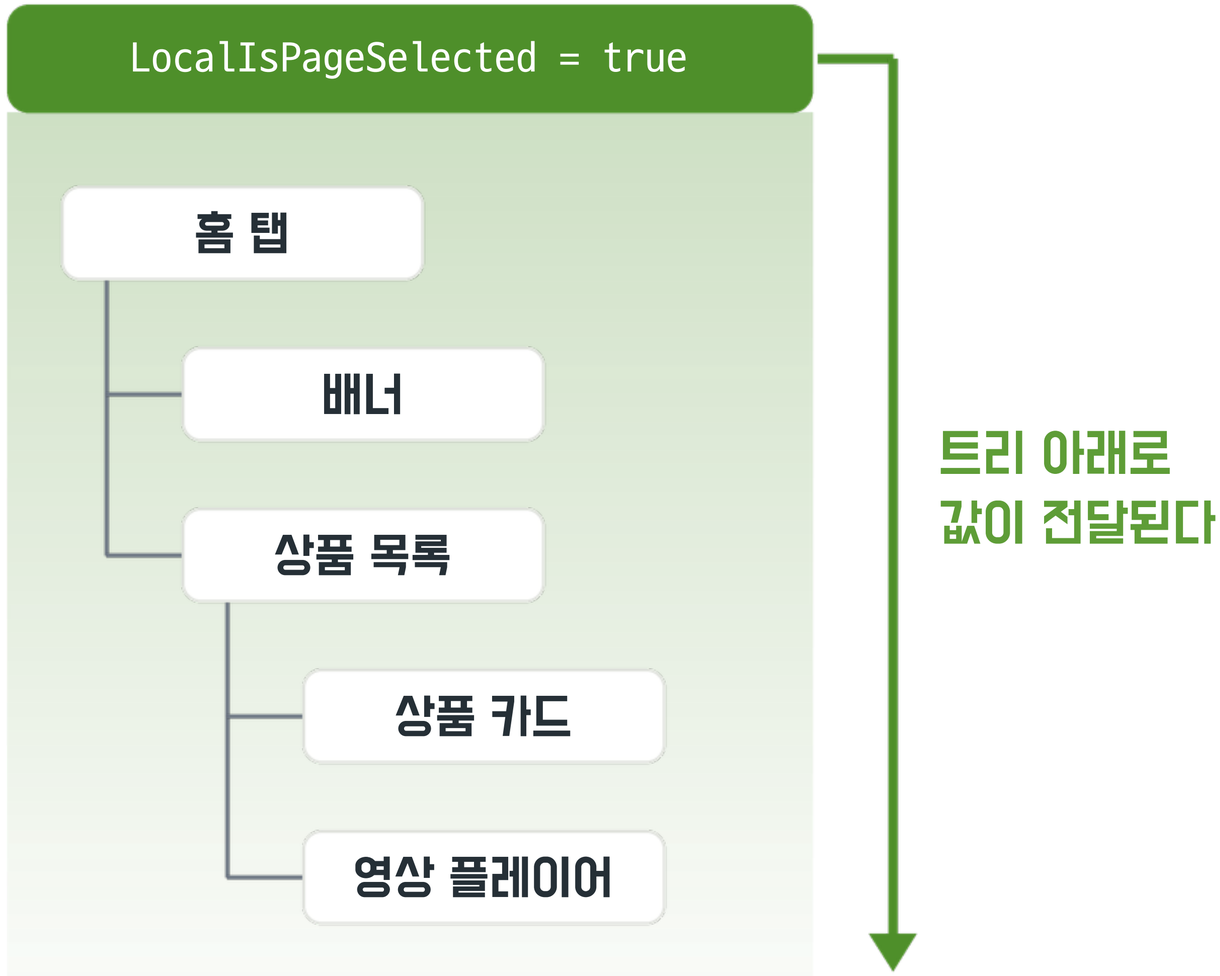
compositionLocalOf

UI 트리 전체에 값을 내려 주는 전역 값 같은 장치

여기에 넣어 둔 값

지금 이 탭이 선택되었는가?

true / false



03 사례 ③ 한 줄 수정

전역 값 하나가 화면 전체를 다시 그리다

Before

두 탭 전체가 다시 실행

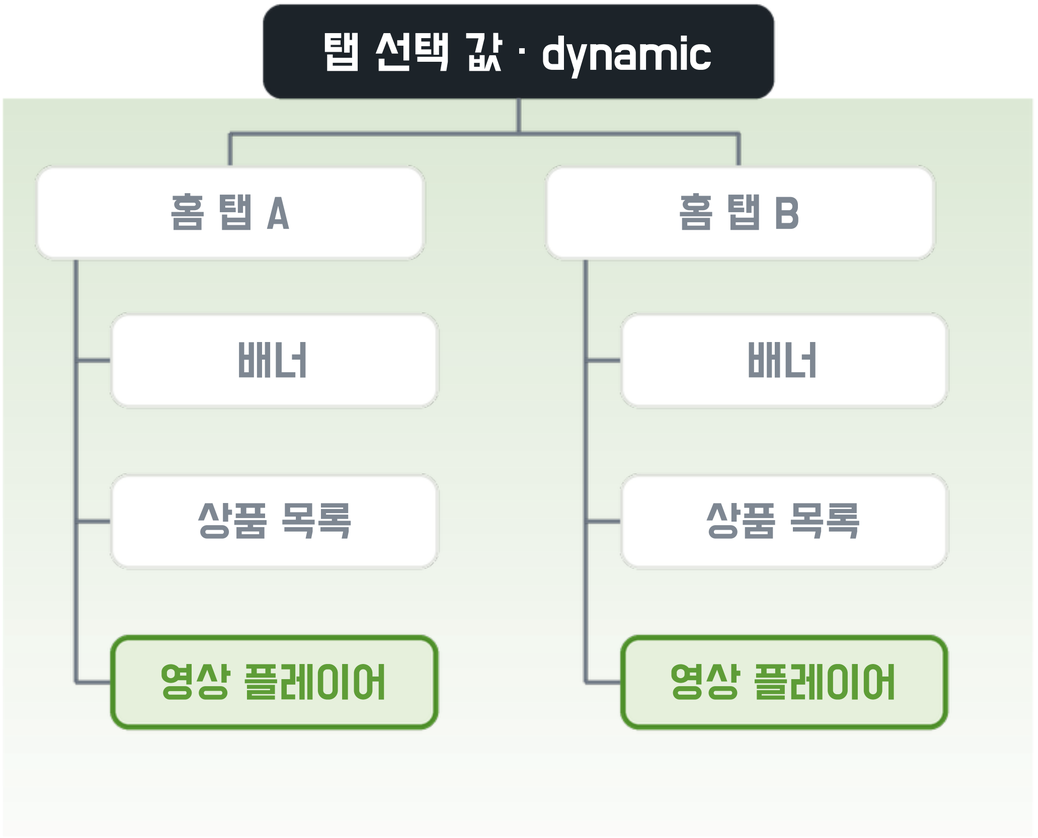
```
val LocalIsPageSelected = staticCompositionLocalOf { true }
```



After

영상 플레이어만 다시 실행

```
val LocalIsPageSelected = compositionLocalOf { true }
```

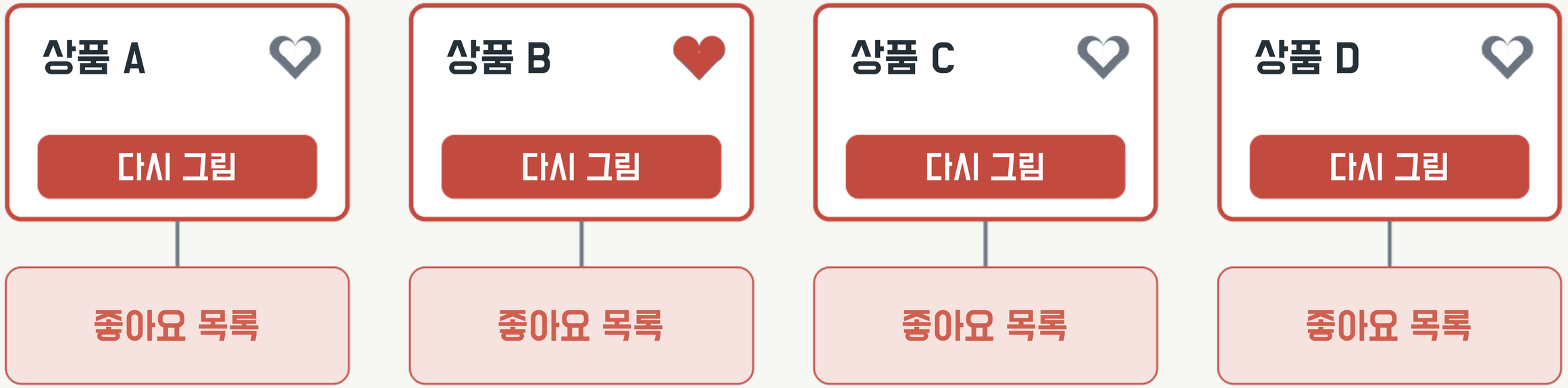


03 사례 ④ 구독 범위

하트 하나 눌렀는데, 카드 전부가 다시 그려진다

Before · 하트 1개 → 카드 4장 모두 다시 그림

```
@Composable
fun ProductCard(goodsNo: String, likedList: List<String>) {
  val isLiked = goodsNo in likedList
  Heart(isLiked)
}
```

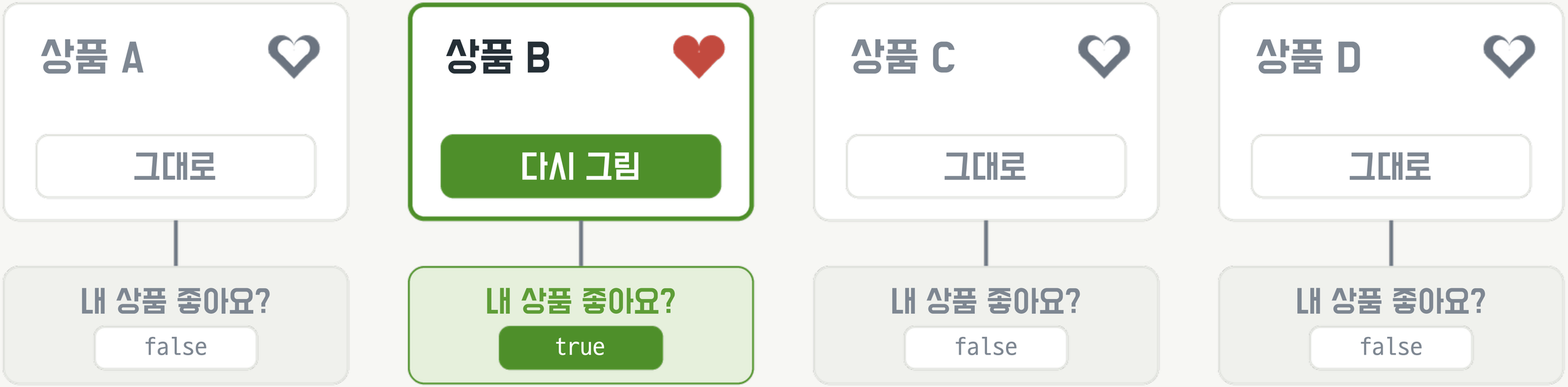


03 사례 ④ 구독 범위

카드마다 '내 상품' 값 하나만 본다

After · 하트 1개 → 누른 카드만 다시 그림

```
@Composable
fun ProductCard(goodsNo: String) {
  val isLiked by rememberIsLiked(goodsNo)
  Heart(isLiked)
}
```



03 사례 ⑤ 안정성

같은 내용을 다시 받아도, 다시 그린다

Before · @Immutable 없음

```
data class Product(val name: String, val tags: List<String>)
```

① API 응답

Product

name = "비타민 C"

tags = ["건강"]

주소 @a1

처음 받은 객체

② 같은 내용을 다시 받음

Product

name = "비타민 C"

tags = ["건강"]

주소 @b2

새로 만든 객체

③ Compose가 비교

같은 객체인가?

@a1 ≠ @b2

내용은 보지 않는다

다르다

④ ProductCard()

다시 실행

내용이 같은데도 다시 그린다

03 사례 ⑤ 안정성

'안 바뀌었다'는 걸 Compose에게 알려 준다

After · @Immutable 사용

```
@Immutable data class Product(val name: String, val tags: List<String>)
// '만들어진 뒤 바뀌지 않는다'고 Compose에게 약속한다
```

① API 응답

Product

name = "비타민 C"

tags = ["건강"]

주소 @a1

처음 받은 객체

② 같은 내용을 다시 받음

Product

name = "비타민 C"

tags = ["건강"]

주소 @b2

새로 만든 객체

③ Compose가 비교

값이 같은가?

name 같음

tags 같음

같다

④ ProductCard()

건너뜀 (skip)

다시 그리지 않는다

04 개념 · remember

다시 실행돼도, 다시 계산하지 않는다

remember 없음

```
@Composable
fun Price(color: Color) {
  val style = TextStyle(color)
  Text("12,900원", style = style)
}
```

Price() 1번째 실행 → style 새로 만든다

Price() 2번째 실행 → style 새로 만든다

Price() 3번째 실행 → style 새로 만든다

style 객체 3번 생성됨

remember 사용

```
@Composable
fun Price(color: Color) {
  val style = remember(color) { TextStyle(color) }
  Text("12,900원", style = style)
}
```

Price() 1번째 실행 → style 새로 만든다

Price() 2번째 실행 → style 그대로 재사용

Price() 3번째 실행 → style 그대로 재사용

style 객체 1개 재사용

04 사례 ① 카운트다운 타이머

1초마다 다시 그리던 타이머

Before · 1초마다 갱신

```
val left by produceState("00:00", endTime) {
  while (true) { ...; delay(1_000) }
}
Text(left, style = timerStyle.copy(color = color))
```

갱신 주기	1초마다
앱이 안 보일 때	계속 갱신
글자 스타일	매번 새로 만든다

1분에 60번 다시 그림

After · 분 단위 갱신 + remember

```
val left by countdownRemainingMinutesFlow(endTime)
  .collectAsStateWithLifecycle(minActiveState = RESUMED)
val style = remember(color) {
  timerStyle.copy(color = color)
}
Text(left, style = style)
```

갱신 주기	분이 바뀔 때만
앱이 안 보일 때	멈춘다
글자 스타일	remember로 재사용

1분에 1번 다시 그림

04 사례 ② 반복 생성 제거

호출할 때마다 만들던 정규식과 포맷

Before · 부를 때마다 생성

```
fun Int.toFormattedNumber() =
  DecimalFormat("#,###").format(this)
```

After · 한 번 만들어 재사용

```
private val groupedNumberFormat =
  ThreadLocal.withInitial { DecimalFormat("#,###") }
fun Int.toFormattedNumber() =
  groupedNumberFormat.get().format(this)
```





04 개념 · derivedStateOf

값이 바뀔 때마다 다시 그린다

Before · 매 프레임 값을 그대로 읽는

```
val isDimVisible = dimProgress.va
```



프레임	dimPr	UI 함수
1프레임		처음 그림
2프레임	0	다시 실행
3프레임	0	다시 실행
4프레임	0	다시 실행
5프레임	0	다시 실행
6프레임	1	다시 실행

04 개념 · derivedStateOf

결과가 바뀔 때만 다시 그린다

After · derivedStateOf로 결과만 본다

```
val isDimVisible by remember {
  derivedStateOf { dimProgress.value > 0f }
}
```

프레임	dimProgress	isDimVisible	UI 함수
1프레임	0	false	처음 그림
2프레임	0.2	true	다시 실행
3프레임	0.4	true	그대로
4프레임	0.6	true	그대로
5프레임	0.8	true	그대로
6프레임	1.0	true	그대로

05 사례 ① 로딩 스켈레톤

프레임마다 새로 만들던 그라

Before · 매 프레임 새 Brush

```
Modifier.drawBehind {
  drawRect(Brush.linearGradient(colors, ...))
}
```

가 바뀔 때만 생성

```
WithCache {
 Brush.linearGradient(colors, ...)
d {
(dx) { drawRect(brush) }
```

1프레임 → Brush 새로

2프레임 → Brush 새로

3프레임 → Brush 새로

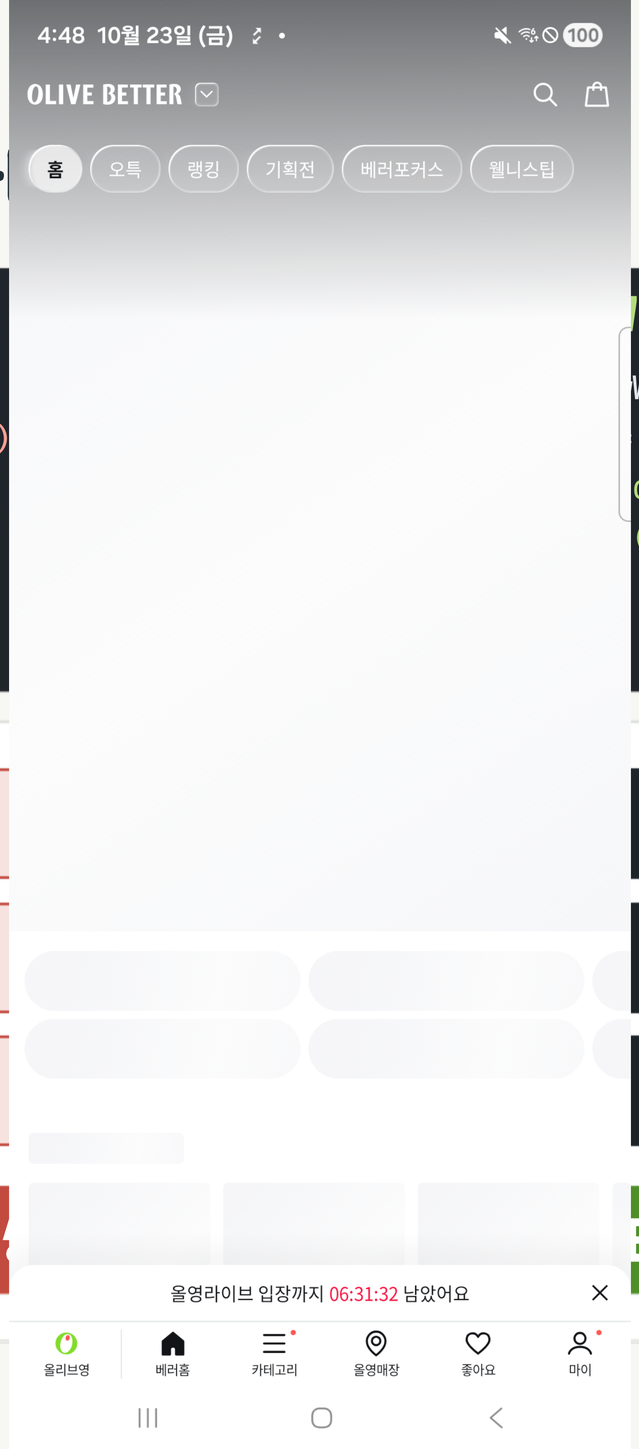
프레임마다 Brush + 셰이더

→ Brush 만든다

→ 위치만 옮긴다

→ 위치만 옮긴다

Brush 1개를 계속 재사용



05 사례 ② 영상 플레이어 화면

보이지 않는 버튼을 매번 만들던 영상 플레이어

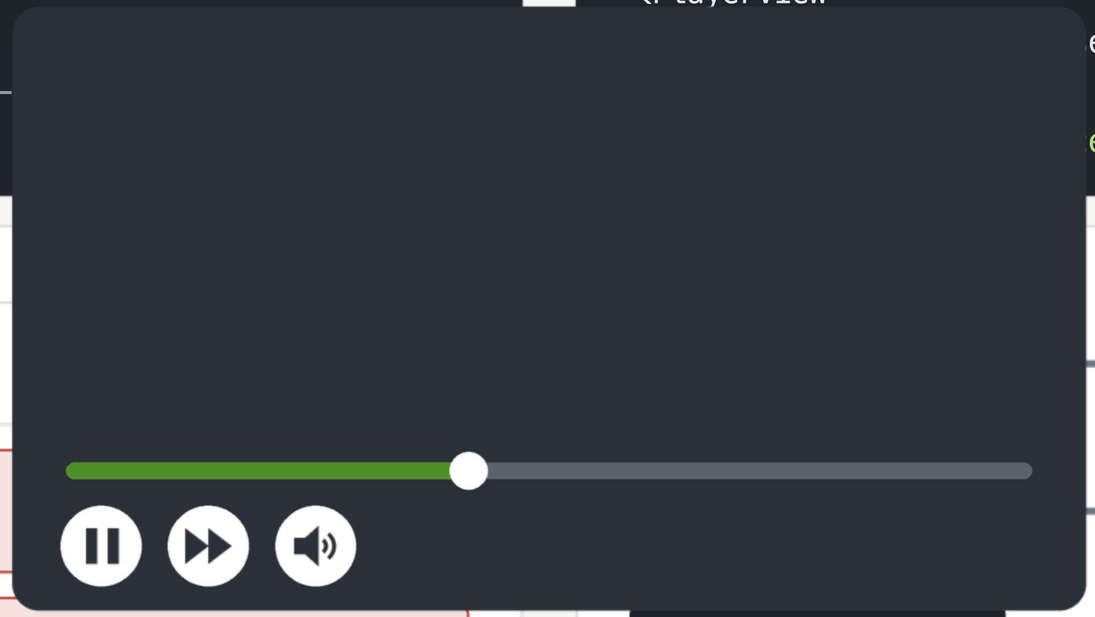
Before · 만들고 숨긴다

```
<PlayerView
  app:use_controller="false" />
<!-- 컨트롤러 자리는 남아 있다 -
```

After · 필요한 것만 만든다

```
<PlayerView
                              e"

                              e" />
```



	Before · 만들고 숨긴다	After · 필요한 것만 만든다
영상 화면	만들고 숨긴다	만든다
재생 버튼 · 진행 바	만들고 숨긴다	만들지 않는다
설정 메뉴 …	만들고 숨긴다	만들지 않는다
	화면 객체 42개 · 약 2ms	화면 객체 9개 · 약 0.2ms

05 사례 ③ 영상 플레이어 풀

영상 플레이어는 버리지 않고 재사용한다

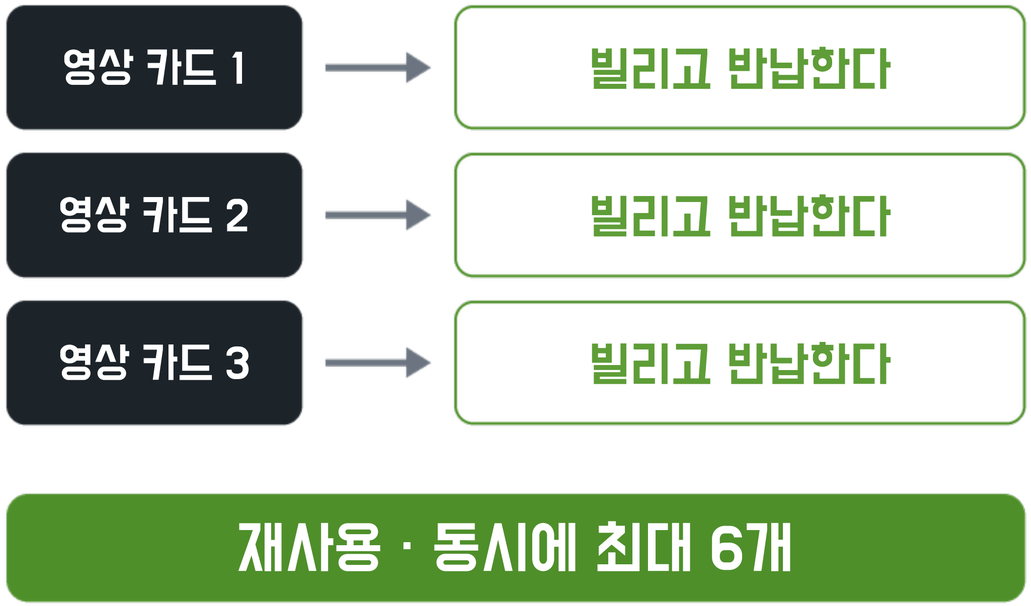
Before · 카드마다 만들고 버린다

```
val player =
  ExoPlayer.Builder(context).build()
// 카드가 사라지면
player.release()
```

After · 풀에서 빌리고 반납

```
val player = pool.acquire(this)
// 동시에 최대 6개 · 카드가 사라지면
pool.recycle(this, player)
// 초기화해서 반납
```





06 체크리스트

내 화면에 바로 대 볼 질문 여섯 개

- 애니메이션 값을 화면 구성 단계에서 읽고 있지 않은가?
  → 그리기·배치 단계에서 읽는다
- 재구성마다 같은 객체를 새로 만들지 않는가?
  → remember · 상수로 한 번만
- 아이템이 목록 '전체'를 구독하고 있지 않은가?
  → 자기 값 하나만 관찰한다
- 보이는 단위보다 자주 갱신하지 않는가?
  → 표시 단위에 갱신 주기를 맞춘다
- 자주 바뀌는 값이 static 전역 값에 있지 않은가?
  → 읽는 곳만 추적하는 방식으로
- 쓰지 않는 화면·객체를 만들고 있지 않은가?
  → 만들지 않거나 재사용한다

06 정리

덜 그리고, 덜 계산하고, 덜 만든다

1

그리는 단계 줄이기

늦게 읽고, 좁게 구독해서
다시 실행되는 범위를 줄인다

2

계산 횟수 줄이기

remember와 상수로
같은 계산을 반복하지 않는다

3

생성 횟수 줄이기

프레임마다 만드는 객체와
안 쓰는 화면을 없앤다

그리고 — 측정하고, 동작은 그대로 둔다

Q & A

들어 주셔서 감사합니다

유준성 · 웰니스서비스개발팀 Android